

以優先權選擇的混合式基因演算法解決旅行家問題

胡哲維¹ 林葭華^{2*}

¹ 國立中山大學資訊工程學系

² 國立中山大學資訊工程學系與通識教育中心

*chlin@cse.nsysu.edu.tw

摘要

近年來由於行動代理者系統的應用廣泛，行動代理者路徑規劃的相關研究逐漸被重視，簡單來說其即為旅行家問題的一種變形，而旅行家問題是眾所熟知的 NP-hard 問題，無法在多項式的時間內求得最佳解。我們認為基因演算法是一個常用於解決最佳化問題的啟發式演算法，將之與簡單的 2-OPT 區域搜尋演算法結合，便可形成效能更佳的混合式基因演算法，並利用旅行家問題中城市分佈隱含的問題特徵知識，將所有的路徑片段賦予不同的兩階層優先權，以分辨基因的優劣，可改善傳統的基因演算法與 2-OPT 區域搜尋演算法，有效縮短搜尋最佳解的時間，因此提出智慧型 OPT 混合式基因演算法 (Intelligent-OPT Hybrid Genetic Algorithm, IOHGA)，並以模擬測試評估 IOHGA 演算法的效能，實驗結果顯示 IOHGA 演算法可在以較短的時間內得到較精確的近似最佳解。

關鍵字：旅行家問題，混合式基因演算法，優先權選擇，2-OPT

1 簡介

旅行家問題 (Traveling Salesman Problem, TSP) 為給定 n 個城市和一個 $n \times n$ 的距離矩陣，試尋找出一條最短路徑，其可以拜訪所有城市恰好一次並回到原出發點。旅行家問題為眾所熟知的 NP-hard 問題，無法在多項式的時間內獲得最佳解 [10]。然而，基因演算法 (Genetic Algorithm, GA) 是一個常見的啟發式演算法，對於旅行家問題而言，可以在合理的時間內得到近似最佳解，其是以生物基因組自然進化的原則為基礎，全域性地搜尋可能解。一般而言，基因演算法包含複製、交配以及突變三個運算子，而將傳統的基因演算法結合區域性搜尋演算法，便可形成強而有力的混合式基因演算法 (Hybrid Genetic Algorithm, HGA) [15, 16, 18]。

2-OPT [4] 是一個以邊為基礎的區域搜尋演算法，於 1957 年由 Croes 所提出，簡單並且廣泛地被運用，運作原理為任意由路徑中取出兩個路徑片段，若交錯重組能夠獲得改善，則將原路徑交錯重組而得到新的路徑。在本文中，將

傳統基因演算法與 2-OPT 區域搜尋演算法結合，形成混合式基因演算法。雖然傳統基因演算法與 2-OPT 區域搜尋演算法都相當簡單，且易於實行，但共同的缺點即為搜尋最佳解的過程較耗費時間，因而需要被改善。

在旅行家問題中，城市的分佈狀況隱含著幾何特性，稱之為問題特徵知識 (Problem-specific Knowledge) [11,23]，可以被利用來提高基因演算法搜尋最佳解的能力。譬如某些路徑線段距離太長而不會出現在最短的路徑上，這樣的資訊可以幫助基因演算法著重於較合適的路徑線段上，避免太過頻繁地考慮距離較長的路徑線段。因此，利用此概念，定義候選集合與非候選集合，並將所有路徑片段依照是否為區域性最佳路徑片段分別置入其中，藉此賦予路徑片段擁有兩階層的優先權。利用優先權等級不同的概念，設計出 (1) 歪斜產生器 (Skewed Production, SP) 產生品質較優良的第一代個體、(2) 優良子路徑交配法 (Fine Subtour Crossover, FSC) 以保留優良的基因給予後代以及 (3) 智慧型 OPT 區域搜尋演算法 (Intelligent OPT, IOPT) 使 2-OPT 搜尋程序執行更有效率；並將三者結合成為的智慧型 OPT 混合式基因演算法 (Intelligent-OPT Hybrid Genetic Algorithm, IOHGA)。

我們並建構了模擬測試以評估 IOHGA 演算法的執行效能，實驗結果顯示，如果在不強調必須找到最佳解的情況下，IOHGA 演算法可以使用少量的時間，而提供較高精確度的近似最佳解。

在以下的章節中，將在第二節說明為何以區域性最佳路徑作為候選集合與非候選集合分類的準則，所提出的 IOHGA 將在第三節中詳細地描述，而效能分析與實驗結果則顯示於第四節，最後則在第五節作結論。

2 最短路徑片段

在旅行家問題中，城市的分佈情形包含著一些幾何上的特性，能夠被利用來評估一個路徑片段出現於最佳路徑中的可能性，而這些幾何特性即所謂的問題特徵知識 (Problem-specific

* 通訊作者

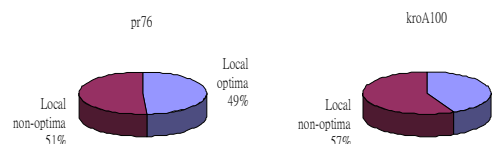


圖 1 pr76 與 kroA100 TSPLIB 測試檔案最佳路徑組成成分分析圖

Knowledge) [11,23]。由觀察得知，在旅行家問題中，大部分的路徑片段太長，而不會出現於最佳路徑，可以省略不必計入考慮，因此，只需要將注意力集中於有機會的候選路徑片段即可，如此便可縮短獲得最佳解的搜尋時間。

根據上述的觀念，所有的路徑片段可以被分割成兩個獨立的集合，一個稱之為候選集合，另一個則稱之為非候選集合。在候選集合中的路徑片段，將被認定為候選優質路徑片段，其餘則為非候選優質路徑，置於非候選集合中。何者為候選優質路徑片段？由於路徑片段中，

長度短者比長度長者出現於最佳路徑的機率高，因此，遴選長度短者作為候選優質路徑片段，但不同旅行家問題路徑長度的長短將難以界定。根據對於 pr76 與 kroA100 兩個 TSPLIB 測試檔案的分析，如圖 1 所示，pr76 最佳路徑的路徑組成成分中，含有 37 個區域性最佳路徑片段，佔有率為 49%；而在 kroA100 最佳路徑路徑組成成分中，含有 43 個區域性最佳路徑片段，佔有率為 43%。因此，以區域性最佳解作為候選優質路徑片段，不僅能夠解決路徑長短難以界定的問題，亦可利用區域性最佳解縮短獲得全域性最佳解的搜尋時間。

對於每一個城市而言，將區域性最佳路徑片段定義為候選優質路徑片段，稱之為最小路徑片段 (Minimal Tour Segments, MTSs)，因此，若 $S_{i,j}$ 為由城市 i 出發的所有路徑片段中，擁有最短長度者，則 $S_{i,j}$ 即為一個 MTS，其餘者則為非最小路徑片段 (Non-minimal Tour Segments, NMTSs)。

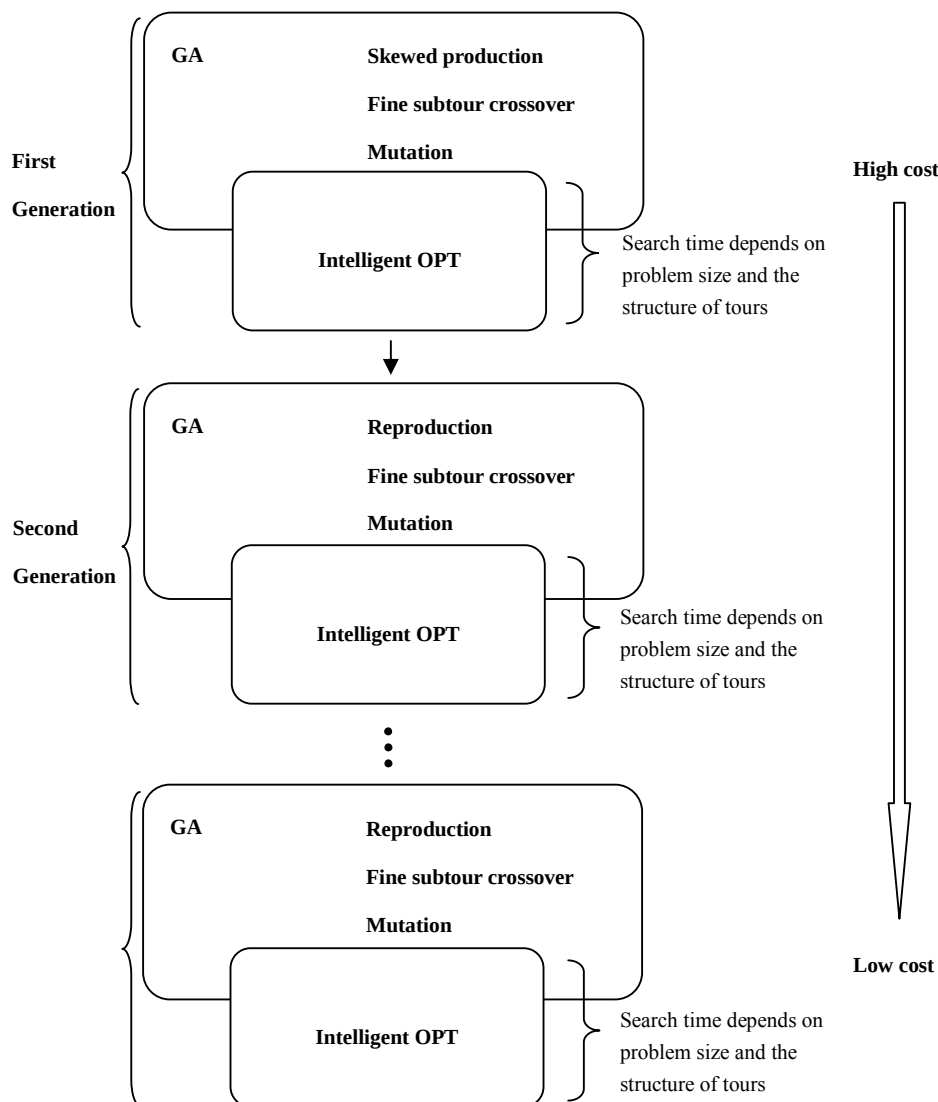


圖 2 IOHGA 架構圖

3 智慧型 OPT 混合式基因演算法

在本篇文章當中，將提出智慧型 OPT 混合式基因演算法 (Intelligent-OPT Hybrid Genetic Algorithm, IOHGA) 以解決旅行家問題。一般來說，在傳統基因演算法中，第一個世代之個體是隨機產生，由世代中挑選出父母進行交配產生下一代的新個體，新個體中也許部分將發生基因突變，最後新個體與原世代個體混合並以適應度遴選優良的個體留至下一世代中，以此模式不斷地世代交替，而模擬自然界的演化過程。在所提出的 IOHGA 演算法當中，將以歪斜產生器 (Skewed Production, SP) 嘗試產生較短的路徑，作為第一世代之個體，並且以優良子路徑交配法 (Fine Subtour Crossover, FSC)，作為基因演算法的交配運算子，企圖將值得傳遞給下一代個體之子路徑保留，最後以智慧型 OPT 區域搜尋演算法 (Intelligent OPT, IOPT) 與之結合形成混合式基因演算法，雖然 2-OPT 區域搜尋演算法相當簡單，但是以隨機方式搜尋解空間，不具智慧而耗費時間，因此，對 2-OPT 演算法做些微修正以更有效率的方式獲得可能解，而 IOHGA 演算法其架構圖如圖 2 所示。於 3.1 節中，將描述 SP 如何產生長度較短的路徑，而 FSC 將在 3.2 節中介紹，最後 IOPT 將被敘述於 3.3 節中。

3.1 歪斜產生器

一般而言，是以隨機方式產生基因演算法中第一世代之個體，如此個體將均勻地分佈於旅行家問題的解空間中，可以避免基因演算法陷入區域性最佳解，而遭遇過早同化的問題，但需要較多的時間演化，同時同化的問題也仍然存在而無法被解決。因此，以較具智慧的方式產生第一個世代之優質個體，能夠節省許多的搜尋時間。在此小節中，將基於幾何特性，利用候選集合的概念，提出歪斜產生器 (Skewed Production, SP) 以產生優質的路徑，加速搜尋的過程。以隨機方式與 SP 產生之路徑分佈狀況如圖 3 所示。

世代中個體的品質是由適應度函式 $f()$ 評估，其定義如公式 (1) 所示：

$$f(T) = 1/c(T) \quad (1)$$

其中 T 為一完整的路徑，而 $c(T)$ 為路徑 T 的長度。相對於隨機方式產生之路徑無法確定個體的品質，由 SP 產生之路徑，其平均長度將是比較短的，路徑長度越短，則其成為最佳解的機率越高。為了產生較短長度的路徑，必須決定哪個路徑片段應該被選擇以組成完整的路徑，根據第二節中對於候選集合與非候選集合的討論，最小路徑片段 (Minimal Tour Segments, MTSs) 是區域性最佳路徑，而且對於路徑適應度的貢獻比非最小路徑片段 (Non-minimal Tour Segments, NMTSs) 大，因此，優先考慮以 MTS

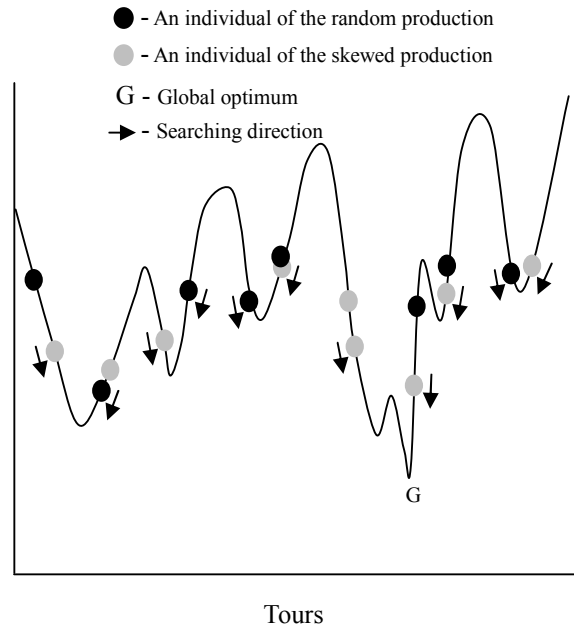


圖 3 以隨機方式與歪斜產生器產生之路徑分佈狀況分佈圖

組成一個完整的路徑，若 MTS 無法組成一個完整的路徑，則將剩餘的部分以 NMTS 完成之。

最佳解的路徑，可能是由所有的 MTS 組合而成，如果無法直接由 MTS 組合而成，則將包含一部份的 NMTS，在這般情況之下，所有 MTS 中，何者應該被挑選出來作為新個體當中的基因呢？因為對於最佳路徑的資訊一無所知，直接產生最佳路徑的複雜度過高，而且目標只是希望產生較優質的第一代個體，最佳解能夠透過基因演算法演化的過程取得，因此，為了避免解空間的搜尋範圍過於狹窄，SP 將隨機挑選 MTS 作為新個體的基因，若無法以 MTS 組成完整的路徑，則利用 NMTS 補滿形成完整路徑。

當需要以 NMTS 完成整個路徑時，哪個 NMTS 應該被選擇呢？將利用一個目標函式 $o()$ 評估 NMTS，以判定哪個 NMTS 值得作為基因，加入至路徑中。因此，SP 的執行步驟如下：

1. 產生一個空路徑 T 。
2. 從剩下的 MTS 中，隨機挑選一個 MTS 填入 T 中，直至所有的 MTS 皆已被考慮為止。
3. 如果路徑 T 並不完整，則由尚未連接完成的路徑中，找出所有可能的 NMTS，並以目標函式 $o()$ 挑選出最佳者，以其連接兩條不完整的路徑，直至路徑 T 為一條完整的路徑為止。

目標函式 $o()$ 之定義如公式 (2) 所示：

$$o(S_{i,j}) = c(S_{i,j})^2 / \min_{x \in N} c(S_{i,x}) \quad (2)$$

其中 i 與 j 皆屬於 N ，而 N 為包含所有城市之集合， $S_{i,j}$ 為由城市 i 出發至城市 j 之路徑片段， $c(S_{i,j})$ 為 $S_{i,j}$ 之長度。

在目標函式 $o()$ 中，將以 $c(S_{i,j})/\min_{x \in N} c(S_{i,x})$ 考量路徑片段 $S_{i,j}$ 與由城市 i 出發之最小路徑片段間的差距程度，同時以 $c(S_{i,j})$ 考量路徑片段 $S_{i,j}$ 之長度，根據此二者之乘積決定目標函式 $o()$ 之值。長度小者，對於路徑之適應度貢獻較大，而與區域性最佳路徑片段差距小者，顯示其近似區域性最佳路徑片段，值得被優先選擇。因此，目標函式 $o()$ 之值越小者，則該 NMTS 越值得被選擇置入未完成的路徑中。依據目標函式 $o()$ 的定義，SP 將比較 NMTS 之間與其個別區域性最佳路徑片段的差距程度，以及長度，當 NMTS 其個別與區域性最佳路徑片段差距程度相同時，則選擇長度短者以連接未完成之路徑；而當 NMTS 其長度相同時，則選擇與其區域性最佳路徑片段差距小者。

3.2 優良子路徑交配法

在現實環境中，生物藉由交配使基因混合與重組，而產生不同的基因組合，優良的基因亦是透過自然的選擇方式遺傳給予後代，因此，生物的演化過程相當緩慢，為了加速生物演化的時程，人類利用基因改造技術來保留需要的基因，所以如果能夠判定何者為優良基因，並維持將其保留給予後代，便可加速演化的過程。在此小節中，將以此為目標，設計優良子路徑交配法 (Fine Subtour Crossover, FSC)。

最小路徑片段 (Minimal Tour Segments, MTSs) 為區域性最佳路徑片段，相較於非最小路徑片段 (Non-minimal Tour Segments, NMTSs) 而言，擁有較大的可能性成為最佳解中的基因，因此，MTS 應該具有較高的優先權被保留給予後代。除 MTS 之外，NMTS 亦有可能成為最佳解中的基因，但何者才是真正值得保留給予後代的路徑片段？完整子路徑交換交配法 (Complete Subtour Exchange Crossover, CSEX) [16,18] 中提出在演化的過程中，較優質的子路徑將慢慢被保留下來，而遺傳給後代，因此，隨著可行解不斷地進化，世代中的個體間，相同的子路徑將越來越多，顯示這些相同的子路徑越來越有可能成為最佳解的基因，但是演化依然需要耗費時間。

在基因演算法演化的初期，路徑之間的結構差異大，相同路徑片段的數量少，在此其間，主要是利用幾何特性提供的資訊，以評斷路徑片段是否值得遺傳給予後代，除了相同的 MTS 被優先考慮之外，不同的 MTS 亦應該被優先考慮以擴大解空間被搜尋的範圍，隨著演化的過程，路徑之間的結構，將越來越相似，世代中路徑之間，相同的路徑片段越來越多，而不同的 MTS 越來越少，因此，可藉由路徑結構上所獲得的額外資訊，以判定路徑片段是否值得被

遺傳給予後代。將此兩個資訊結合便可形成四個選擇規則的優先順序如下所示：

1. 相同的 MTS。
2. 相異的 MTS。
3. 相同的 NMTS。
4. 相異的 NMTS。

此四個優先順序中，利用前三個優先規則，便可從參與交配的個體間保留值得遺傳給予後代的路徑片段，在前三個優先規則後，若無法形成一個完整路徑，則由 NMTS，並利用 3.1 節所述的目標函式 $o()$ 遴選，以補滿形成完整的路徑。因此，FSC 的執行步驟如下：

1. 產生一條空路徑 T 。
2. 將父母個體間所有相同之 MTS，一一填入 T 中。
3. 將父母個體間所剩下相異之 MTS，依照最短路徑優先的方式，一一填入 T 中。
4. 如果路徑 T 並不完整，則挑選父母個體間，相同之 NMTS 者，依照目標函式 $o()$ 值最小者優先的方式，一一填入 T 中。
5. 如果路徑 T 尚不完整，則由尚未連接完成的路徑中，找出所有可能的 NMTS，並以目標函式 $o()$ 挑選出最佳者，以其連接兩條不完整的路徑，直至路徑 T 為一條完整的路徑為止。

FSC 與 CSEX 事實上非常相似，在目的上，兩者都一樣是尋找值得遺傳給予後代的路徑片段，但 FSC 更強調 MTS 較 NMTS 擁有更高的優先權被遺傳給予下一代，路徑片段有優先權的區別，而 CSEX 對於所有的路徑片段一視同仁。

3.3 智慧型 OPT 區域搜尋演算法

2-OPT 區域搜尋演算法是一個以路徑片段為基礎的區域搜尋演算法，由 Croes 於 1958 年提出，用以解決旅行家問題。2-OPT 已有許多的延伸版本，例如：LK 區域搜尋演算法[19]等，雖然它們確實改善 2-OPT 區域搜尋演算法，但亦同時增加其演算法的複雜度，在此小節中，將提出智慧型 OPT 區域搜尋演算法 (Intelligent OPT, IOPT)，不僅改善 2-OPT 區域搜尋演算法的搜尋過程，亦同時維持 2-OPT 區域搜尋演算法的簡易。

2-OPT 區域搜尋演算法概念相當地簡單，為任意挑選兩個路徑片段，嘗試將之交錯重組，先看是否能夠獲得更短長度的可行解，如果長度縮短，便確實執行交錯重組原路徑，稱為

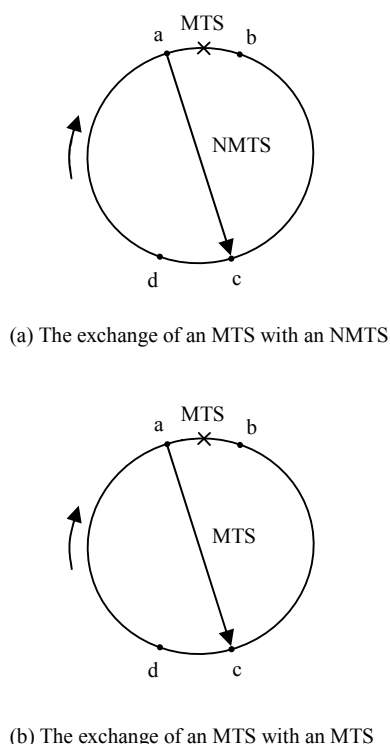


圖 4 2-OPT 區域搜尋演算法搜尋過程中，不適當之交換

2-OPT 交換，以獲得新的可行解。每一次任意挑選兩個路徑片段後，皆必須花費時間去測試其交錯重組後是否可以改善原路徑，但選擇與測試將可能是徒勞無功，而浪費時間在執行無意義的事情，因此，有方向性的選擇將比任意式的選取更有效率。因此，IOPT 將引導 2-OPT 搜尋程序執行朝正確的方向進行搜尋，以縮短搜尋時間。

由於任何路徑皆是最小路徑片段 (Minimal Tour Segments, MTSs)，以及非最小路徑片段 (Non-minimal Tour Segments, NMTSs) 共同組成，幾何特徵可以被利用來引導 2-OPT 區域搜尋演算法的搜尋過程，使其更具智慧。由觀察得知，若原先存在於路徑當中者為一 MTS，而以一 NMTS 取代之，則只會增加路徑之長度，如圖 4a 所示；若原先存在於路徑當中者為一 MTS，而以另一 MTS 取代之，亦無法縮短路徑之長度，如圖 4b 所示，由此可知，NMTS 較適合被嘗試著以 MTS 或是其他 NMTS 取代。因此，IOPT 將不斷地尋找最應該先被取代的 NMTS，並嘗試將縮短的長度最大化，如此便可有效改善 2-OPT 區域搜尋演算法，以增快搜尋可行解的過程。

何者是最應該被取代的 NMTS？假設僅考慮城市 i ，則當 $S_{i,j}$ 為一個 NMTS，其與由城市 i 出發之 MTS 差距越大，則其在取代之後，原路徑可能獲得的改善越大，因此，比較各個 NMTS 與其個別 MTS 之差距程度，即可得知何者為最應該被取代的 NMTS。在 IOPT 中，將以目標函式 $w()$ ，來評估 NMTS 是否值得被取代，而目

標函式 $w()$ 定義如公式 (3) 所示：

$$w(S_{i,j}) = c(S_{i,j}) / \min_{x \in N} c(S_{i,x}) \quad (3)$$

其中 i 與 j 皆屬於 N ，而 N 為包含所有城市之集合， $S_{i,j}$ 為由城市 i 出發至城市 j 之路徑片段， $c(S_{i,j})$ 為 $S_{i,j}$ 之長度。

IOPT 之執行步驟如下：

1. 判定路徑中是否有 NMTS 需要被取代，若無則終止 IOPT。
2. 以目標函式 $w()$ 評估所有的 NMTS，具有最大 $w()$ 值的 NMTS，則優先被替換，以期縮短原路徑的長度。
3. 選擇另一路徑，此路徑可使得執行 2-OPT 交換後，路徑長度能夠獲得最大幅度的改善。
4. 執行 2-OPT 交換，回到步驟 1。

4 實驗結果

在下列實驗中，是執行於 80*86 桌上型電腦，CPU 為 Celeron 2.8GHz 與 480MB RAM 環境下，分別以不同的參數，將智慧型 OPT 混合式基因演算法 (Intelligent-OPT Hybrid Genetic Algorithm, IOHGA) 與 Katayama 提出的混合式基因演算法之退火法 (Hybird Genetic Algorithm with Simulated Annealing Algorithm, HGA(SA)) [18] 以及 Burkowski 提出的基因表示演算法 (Gene Expression Algorithm, GEGA) [3] 比較。其中誤差率的計算方式皆定義如公式 (4) 所示：

$$Error\ Rate = \frac{Cost_{obtained} - Cost_{optimal}}{Cost_{optimal}} \times 100(\%) \quad (4)$$

4.1 實驗 1

此實驗是將 IOHGA 與 HGA(SA) 比較，其以完整子路徑交換交配法 (Complete Subtour Exchange Crossover, CSEX) 作為交配運算子，並以退火法 (Simulated Annealing Algorithm) 作為區域搜尋演算法。實驗中將測試 14 個 TSPLIB 測試檔案 [23]，並以最終解之最小誤差、最大誤差、平均誤差、10 回合中獲得最佳解的次數 (Opt/10 Runs) 以及 CPU 執行時間以評估效能，個體數將被設定為 10，交配率為 100%，世代數為 200。

由於 HGA(SA) 之執行環境為 S-4/5 工作站 (microSPARC II, 100MHz)，與 IOHGA 執行環境不同，因此，在表 1 裡，針對 HGA(SA) 的執行環境計算出對應於 IOHGA 執行環境的 CPU 執行時間對應值，相對應的 CPU 執行時間為原始 CPU 執行時間乘以 110，再除以 2800。實驗的結果如表 1 與 2 所示。

表 1 HGA(SA)之實驗結果

TSPLIB Instance	HGA(SA)(%)			Opt / 10 Runs	Original CPU Time(s)	Relative CPU Time(s)
	min.	max.	avg.			
kroA100	0.00	0.38	0.12	7	unknown	unknown
kroA150	0.00	2.35	1.36	4	unknown	unknown
kroA200	0.33	2.36	1.21	0	806	31.66
lin105	0.00	0.41	0.04	9	232	9.11
lin318	0.53	2.86	1.80	0	1233	48.44
pr76	0.00	0.86	0.24	7	118	4.63
pr107	0.00	0.40	0.04	9	130	5.12
pr124	0.00	1.37	0.19	5	126	4.95
pr152	0.00	0.82	0.29	6	361	14.18
pr226	0.01	0.84	0.32	0	840	33.00
pr264	0.00	1.28	0.42	3	1029	40.43
pr299	0.47	2.33	0.95	0	1537	60.38
pr439	0.31	3.34	1.36	0	1805	70.91
rd100	0.01	0.70	0.12	0	unknown	unknown

表 2 IOHGA 之實驗結果

TSPLIB Instance	HGA(SA)(%)			Opt / 10 Runs	CPU Time(s)
	min.	max.	avg.		
kroA100	0.00	0.38	0.12	7	1.83
kroA150	0.00	0.63	0.17	4	11.88
kroA200	0.02	2.66	0.96	0	23.00
lin105	0.00	0.15	0.01	9	0.95
lin318	0.32	1.29	0.85	0	62.77
pr76	0.00	0.10	0.01	7	0.67
pr107	0.09	0.55	0.34	0	4.70
pr124	0.00	0.09	0.01	9	1.73
pr152	0.00	0.26	0.14	3	8.95
pr226	0.00	0.23	0.09	1	15.33
pr264	0.00	0.76	0.21	3	15.89
pr299	0.39	1.38	0.86	0	50.47
pr439	0.29	0.95	0.71	0	111.50
rd100	0.00	0.03	0.00	7	1.89

在 HGA(SA)中,大部分的 CPU 執行時間是花費在區域搜尋演算法,因此,其採用數量少的個體數,以降低 CPU 執行時間,但對於基因演化而言,將造成基因演算法快速地遭遇到同化

問題,然而由於退火法搜尋解空間的方式為隨機,因此,同化問題對於搜尋解空間的過程影響並不甚巨,但是相對於 IOPT 而言,搜尋過程是根據輸入的路徑組成結構,當 IOHGA 遭遇同化問題時,輸入 IOPT 之路徑便完全一樣,如此小數量的個體數對於 IOHGA 有相當大的影響,若增加個體數必定能夠提高獲得最佳解的次數。

4.2 實驗 2

此實驗是將 IOHGA 與 GEGA 比較,其以利用插入法為基礎之建構演算法 (Construction Heuristic) 產生第一代的個體,在文章中,共提及四種插入策略,但是卻沒有指明於實驗時,是採取哪一種插入策略,作為建構演算法的基礎,除了利用建構演算法產生第一代個體之外,其以 inver-over 運算子改善路徑的長度。實驗中將測試 8 個 TSPLIB 測試檔案[23],並以最終解之最小誤差以及 CPU 執行時間以評估效能,個體數將被設定為 256 與 1024 兩組,交配率為 100%,沒有設定世代數,因為終止條件為連續兩個世代無法產生更優良的個體即停止。

由於 GEGA 之執行環境為 Intel Pentium III 600MHz,與 IOHGA 執行環境不同,因此,在表 3 與 5 裡,針對 GEGA 的執行環境計算出對應於 IOHGA 執行環境的 CPU 執行時間對應值,相對應的 CPU 執行時間為原始 CPU 執行時間乘以 600,再除以 2800。實驗的結果如表 3、4、5 以及 6 所示。

由兩組實驗結果可得知,當個體數增加時,誤差率也會同時減小,因為搜尋空間的覆蓋率增加,基因組合的變化較多,因而更有機會獲得最佳解[12]。此外,實驗當中停止條件的設定,對於傳統基因演算法的基因演化過程而言,過於嚴苛,雖然連續兩個世代都無法獲得更優質的路徑,但可能路徑依然有機會於後幾代中獲得改善,因此,若以較大的世代數設定之,必定可以獲得更好的結果。

表 3 GEGA 採用個體數為 256 之實驗結果

TSP Instance	Opt. Length	Tour Length	Min. Error Rate (%)	Original CPU Time(s)	Relative CPU Time(s)
berlin52	7542	7718	2.33	12	2.57
Ch130	6110	6336	3.70	49	10.5
Ch150	6528	6910	5.85	89	19.07
Eil51	426	432	1.40	9	1.93
Eil101	629	647	2.86	40	8.57
kroA100	21282	22043	3.58	26	5.57
Lin105	14379	14747	2.56	43	9.21
Tsp225	3919	4002	2.12	420	90.00

表 4 IOHGA 採用個體數為 256 之實驗結果

TSP Instance	Opt. Length	Tour Length	Min. Error Rate (%)	CPU Time(s)
berlin52	7542	7542	0.00	0.57
ch130	6110	6327	3.55	3.61
ch150	6528	6658	1.99	5.00
eil51	426	426	0.00	0.05
eil101	629	640	1.75	2.01
kroA100	21282	21936	3.07	2.20
lin105	14379	14522	0.99	2.44
tsp225	3919	4039	3.14	11.56

合優先權等級的不同，設計歪斜產生器 (Skewed Production, SP)、優良子路徑交配法 (Fine Subtour Crossover, FSC) 以及智慧型 OPT 區域搜尋演算法 (Intelligent OPT, IOPT)，並將之結合成為智慧型 OPT 混合式基因演算法 (Intelligent-OPT Hybrid Genetic Algorithm, IOHGA)。

經由 TSPLIB 測試檔案實驗後，結果數據顯示 IOHGA 擁有不錯的效能，雖然無法保證永遠獲得最佳解，但所獲得的近似最佳解，平均誤差小，而且花費的時間並不多，在不強調最佳解的情況之下，IOHGA 能夠以省時且精確的方式提供近似最佳解。因此，IOHGA 可能可以與某些叢聚演算法結合，並運用至在及時環境下的行動代理者規劃問題當中。

表 5 GEGA 採用個體數為 1024 之實驗結果

TSP Instance	Opt. Length	Tour Length	Min. Error Rate (%)	Original CPU Time(s)	Relative CPU Time(s)
berlin52	7542	7542	0.00	39	8.36
Ch130	6110	6227	1.91	483	103.50
Ch150	6528	6711	2.80	503	107.79
Eil51	426	426	0.00	34	7.29
Eil101	629	641	1.91	169	36.21
kroA100	21282	21831	2.58	199	42.64
Lin105	14379	14379	0.00	266	57.00
Tsp225	3916	3952	0.92	2745	588.21

表 6 GEGA 採用個體數為 1024 之實驗結果

TSP Instance	Opt. Length	Tour Length	Min. Error Rate (%)	CPU Time(s)
berlin52	7542	7542	0.00	2.12
ch130	6110	6303	3.16	13.78
ch150	6528	6667	2.13	20.30
eil51	426	426	0.00	2.00
eil101	629	637	1.27	8.16
kroA100	21282	21715	2.03	8.50
lin105	14379	14401	0.15	9.30
tsp225	3916	4016	2.55	45.14

參考文獻

- [1] W. Banzhaf. The “Molecular” Traveling Salesman. *Biological Cybernetics*, 64: 7–14, 1990.
- [2] T. Back. The interaction of mutation rate, selection, and self-adaptation within a genetic algorithm. In R. Miinner and B. Manderick, editors, *Parallel Problem Solving 2*, Amsterdam, Elsevier, 1992.
- [3] F. J. Burkowski. Proximity and priority: applying a gene expression algorithm to the Traveling Salesperson Problem. *Parallel Computing*, Vol. 30, pp. 803–816, May 2004.
- [4] G. A Croes. A Method for Solving Traveling Salesman Problems. *Operations Research*, Vol. 6, No. 6, pp. 791–812, 1958.
- [5] E. G. David. Genetic Algorithms in Search, Optimization, and Machine Learning. *Addison Wesley*, pp. 1–88, 1989.
- [6] M. Englert, H. Röglin, and B. Vöcking. Worst Case and Probabilistic Analysis of the 2-Opt Algorithm for the TSP. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM, Philadelphia, PA, pp. 1295–1304, B. 2007.
- [7] D. B. Fogel. An Evolutionary Approach to the Traveling Salesman Problem. *Biological Cybernetics*, 60: 139–144, 1988.
- [8] D. B. Fogel. A Parallel Processing Approach to a Multiple Traveling Salesman Problem Using Evolutionary Programming. In Canter, L. (ed.) *Proceedings on the Fourth Annual Parallel Processing Symposium*, pp. 318–326, Fullerton, CA, in 1990.
- [9] D. B. Fogel. Applying Evolutionary Programming to Selected Traveling Salesman Problems. *Cybernetics and Systems*, 24: 27–36, 1993.
- [10] M. R. Garey, D. D. Johnson, and R. E. Tarjan. The Planar Hamiltonian Circuit Problem is

5 結論

在本篇文章中，採用兩階層的優先權，將路徑片段劃分為兩個分離的集合，一個集合為候選集合，另一個為非候選集合，利用兩個集

- NP-complete. *SIAM Journal of Computing*, 5:704-714, 1976.
- [11] J. J. Grefenstette. Incorporating Problem Specific Knowledge into Genetic Algorithms. in *Genetic Algorithms and Simulated Annealing*, (L. Davi, ed), pp. 42-60, *Morgan Kaufmann Publishers*, 1987.
 - [12] J. H. Holland *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: Univ. Michigan Press, 1975, 1975.
 - [13] B. A. Julstrom. Coding TSP Tours as Permutations via an Insertion Heuristic. SAC '99, Proc. 1999 ACM Sym. Appl. Comp., ACM Press, pp. 297-301, 1999.
 - [14] Wei Jyh-Da and D. T. Lee. A New Approach to the Traveling Salesman Problem Using Genetic Algorithm with Priority Encoding. In *proc. 2004 IEEE Congress on Evolutionary Computation*, pp. 1457-1464, 2004.
 - [15] T. Kido. Hybrid Searches Using Genetic Algorithms. In: Kitano H, editor, *Genetic algorithm*, Sangyo Tosho, pp. 61-88, 1993.
 - [16] K. Katayama, H. Sakamoto, and H. Narihisa. An Efficiency of Hybrid Mutation Genetic Algorithm for Traveling Salesman Problem. *Proc 2nd Australia-Japan Workshop on Stochastic Models in Engineering, Technique & Management*, Gold Coast, Australia, pp. 294-301, 1996.
 - [17] K. Katayama, H. Hirabayashi, and H. Narihisa. Performance Analysis for Crossover Operators of Genetic Algorithm. *Systems and Computers in Japan*, Vol. 30, No. 2, pp. 20-30, 1999.
 - [18] K. Katayama and H. Narihisa. An Efficient Hybrid Genetic Algorithm for the Traveling Salesman Problem. *Electronics and Communications in Japan*, Part 3, Vol. 84, No. 2, pp. 76-83, 2001.
 - [19] S. Lin and B. W. Kernighan. An Effective Heuristic Algorithm for the Traveling Salesman Problem. *Operations Research*, Vol. 21, No. 2, pp. 498-516, 1973.
 - [20] P. Larrañaga, C. M. H. Kuijpers, R. H. Murga, I. Inza, and S. Dizdarevic. Genetic Algorithms for the Travelling Salesman Problem: A Review of Representations and Operators. *Artificial Intelligence Review* 13: 129-170, 1999.
 - [21] Z. Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs*. Berlin Heidelberg: Springer Verlag, 1992.
 - [22] G. Reinelt. *The Traveling Salesman: Computational Solutions for TSP Applications*. Vol. 840 of *Lecture Notes in Computer Science*, pp. 64-99, Springer-Verlag, 1994.
 - [23] Reinelt G. <http://www.iwr.uni-heidelberg.de/groups/comopt/software/TSPLIB9-5/index.html>
 - [24] G. Syswerda. Schedule Optimization Using Genetic Algorithms. In Davis, L. (ed.) *Handbook of Genetic Algorithms*, pp. 332-349, 1992, New York: Van Nostrand Reinhold.